

CO4 AT3:

R. Deepasri (192511309):

SET 1 – Big Data Reverse Engineering Answers

1. Website Clicks, Mobile Interactions & Transactions

Probable Architecture:

Web/Mobile/Transactions → Kafka → HDFS/Data Lake → Spark → ML/Analytics → Dashboard

- **Storage:** HDFS or cloud data lake for massive distributed storage.
 - **Ingestion:** Apache Kafka, Flume or NiFi for collecting high-volume data.
 - **Batch processing:** Apache Spark or Hadoop MapReduce.
 - **Data separation:**
 - Structured data → relational databases/data warehouse.
 - Semi-structured data → JSON/XML files in a data lake.
 - **Indexing:** Hive tables, partitions and distributed indexes improve query performance.
 - **Stream processing:** Spark Streaming, Apache Flink or Kafka Streams.
 - **Scalability:** Horizontal scaling by adding nodes.
 - **Partitioning:** Data is partitioned by date, region, customer ID or event type.
 - **Fault tolerance:** Data replication and distributed processing prevent data loss.
 - **Visualization:** Power BI, Tableau or similar BI tools connect to the warehouse/analytics layer.
-

2. E-Commerce Platform

Probable Architecture:

User Events → Kafka → Stream Processing → Customer Profile Store → Recommendation Engine → Website/App

- **Event streaming:** Apache Kafka captures product views, carts, payments and checkout events.

- **Customer profiles:** NoSQL databases such as **Cassandra, MongoDB or DynamoDB** can store rapidly changing profiles.
 - **Session management:** Redis can provide fast session storage and caching.
 - **Data synchronization:** Kafka events synchronize clickstream, session and transaction information.
 - **Query engines:** Spark SQL, Presto/Trino or Hive can analyze large datasets.
 - **Machine learning:**
 - Customer segmentation → K-Means
 - Recommendations → Collaborative filtering
 - Purchase prediction → Random Forest/XGBoost
 - **Data merging:** Structured order data is joined with semi-structured browsing and clickstream data using customer/session IDs.
 - **Scalability:** Kafka partitions and distributed database sharding support large traffic.
 - **Fault tolerance:** Replication, retries and checkpointing provide reliable processing.
 - **KPI dashboards:** Spark/SQL analytics feeds Tableau or Power BI for sales, conversion, abandonment and revenue monitoring.
-

3. Healthcare Big Data System

Probable Architecture:

Hospitals/Labs/Wearables → Integration Layer → Data Lake + Healthcare DB → Spark/Flink → ML → Clinical Dashboard

- **Hybrid storage:**
 - Patient records → relational databases.
 - Images → distributed object storage/data lake.
 - Sensor streams → time-series/NoSQL databases.
- **Integration:** Apache NiFi, Kafka and healthcare interoperability standards such as **HL7/FHIR**.
- **Streaming:** Kafka + Spark Streaming/Flink for wearable and monitoring data.
- **Machine Learning:**

- Disease prediction → Random Forest/XGBoost
 - Medical-image analysis → CNN/deep learning
 - Patient risk prediction → classification models
 - **Processing:** Spark performs large-scale patient and population analytics.
 - **Privacy:** Role-based access control and least-privilege access.
 - **Encryption:** Data encrypted during transmission and storage.
 - **Compliance:** Audit logs, consent management, data masking and controlled access support healthcare privacy requirements.
 - **Clinical pipeline:** Data collection → cleaning → integration → feature extraction → ML prediction → clinical decision support.
-

4. Smart City Platform

Probable Architecture:

Sensors/CCTV/Weather/Transport → Edge Gateway → Kafka → Stream Processing → Time-Series/Spatial DB → Analytics → Dashboard

- **Edge computing:** Local gateways process sensor data near the source, reducing latency and network traffic.
- **Messaging:** Apache Kafka or MQTT handles continuous sensor events.
- **Central storage:** HDFS/cloud data lake stores historical city data.
- **Streaming:** Apache Flink or Spark Streaming processes traffic information in real time.
- **Geospatial analytics:** GIS and spatial databases analyze road locations, vehicle movement and congestion zones.
- **Databases:** InfluxDB/TimescaleDB for time-series data; PostgreSQL with PostGIS for location-based queries.
- **Congestion prediction:** Historical traffic + weather + current traffic → ML model → congestion prediction.
- **Batch analytics:** Spark analyzes historical data for long-term urban planning.
- **Real-time analytics:** Streaming engines identify current traffic problems.
- **Security:** Encryption, authentication, RBAC, network security and audit logs.

- **Visualization:** Grafana, Power BI or Tableau can provide live traffic maps, alerts and operational dashboards.
-

5. Healthcare Analytics System

Probable Architecture:

Patient Records + Wearables + Labs + Appointments → ETL → Data Lake/Warehouse → Spark → ML → Healthcare Dashboard

- **Hybrid storage:**
 - Structured medical/appointment data → relational database/data warehouse.
 - Wearable and medical files → NoSQL/data lake/object storage.
- **ETL:** Apache NiFi, Talend or cloud ETL tools extract, clean, transform and integrate data.
- **Interoperability:** HL7 and FHIR enable exchange between hospitals and clinics.
- **Real-time monitoring:** Kafka collects wearable readings; Spark Streaming/Flink analyzes them continuously.
- **Distributed processing:** Apache Spark processes millions of patient records simultaneously for population-level studies.
- **Machine Learning:**
 - Diagnosis risk → classification models
 - Patient deterioration → predictive models
 - Treatment recommendation → recommendation/predictive analytics
- **Security:** Encryption at rest and in transit, RBAC, authentication and secure APIs.
- **Governance:** Audit logs, access monitoring, data masking, consent and retention policies.
- **Analytics flow:**
Data Collection → ETL → Storage → Feature Engineering → ML → Risk Score → Clinical Decision Support